IQGeo Senior Site Reliability Engineer

Focused Interview Preparation: Chaos Engineering • Proactive SRE • Prometheus • Grafana • OpenTelemetry

How to use this pack: Speak the answers naturally. Keep the core answer to roughly 30–60 seconds, then expand only if the interviewer probes. Use your real Azure Synapse / AWS / Kubernetes experience and never claim tools or production work you have not actually done.

High-probability client focus

The JD and interview feedback point toward four areas: (1) chaos engineering and resilience validation, (2) proactive SRE using SLIs/SLOs and predictive signals, (3) Prometheus/Grafana operational monitoring, and (4) OpenTelemetry for distributed traces, metrics and logs.

1. Chaos Engineering & Chaos Testing

Q: What is chaos engineering?

A: I see chaos engineering as deliberately introducing controlled failures into a system to validate that resilience mechanisms actually work. I start with a hypothesis, define the expected steady-state behavior and customer SLOs, run the smallest safe experiment, observe the system, and stop or roll back if risk increases. The important outcome is learning and closing resilience gaps, not simply causing failures.

Q: How is chaos testing different from normal testing?

A: Normal testing usually validates expected behavior under known conditions. Chaos testing deliberately introduces failure conditions such as pod termination, node loss, network latency or dependency failure. I use it to validate assumptions around redundancy, retries, failover, alerting and recovery. It complements functional, performance and security testing rather than replacing them.

Q: How would you design a chaos experiment for an EKS application?

A: I would first define the steady state using availability, error rate and latency SLOs. Then I would choose a controlled failure, for example terminating a small percentage of application pods. I would run it in a non-production environment first, define blast-radius limits and abort conditions, and observe Prometheus, Grafana and OpenTelemetry signals. If recovery is within the expected SLO, I gradually increase the scope.

Q: Give an example of a useful chaos experiment.

A: A practical experiment would be killing one application pod and verifying that Kubernetes recreates it and traffic continues through the Service or load balancer. I would monitor request success rate, latency, pod readiness, restart behavior and user impact. A more advanced experiment could introduce network latency to a dependency and validate timeout, retry and circuit-breaking behavior.

Q: How would you safely run chaos in production?

A: I would use a small blast radius, explicit approval and a defined rollback or stop mechanism. I would target a limited workload or percentage of traffic and avoid peak-risk windows initially. I would define abort conditions around SLO degradation, error rate and customer impact. The experiment should be observable, reversible and documented before execution.

Q: What metrics would you monitor during a chaos experiment?

A: I'd monitor the golden signals: traffic, latency, errors and saturation. For Kubernetes I would also watch pod restarts, readiness, scheduling, node health and resource utilization. Through OpenTelemetry I'd correlate application spans with dependency failures, while Grafana would give the operational view. The key question is whether the customer-facing SLO remains within the expected boundary.

Q: What if chaos testing reveals that the system does not recover?

A: I'd stop the experiment and restore the system first. Then I'd identify the failed resilience assumption: perhaps insufficient replicas, incorrect readiness probes, missing retry controls, a single dependency, or inadequate capacity. I would document the finding as a reliability gap and create a corrective action with an owner and validation plan. I would rerun the experiment after the fix to prove the improvement.

Q: What chaos tools have you used?

A: If asked specifically about a tool I have not used extensively, I would be transparent: I haven't worked extensively with that specific chaos framework, but I understand the engineering approach and have worked with Kubernetes, monitoring and production reliability. I would explain how I would safely design, observe and validate the experiment rather than inventing hands-on experience.

2. Proactive SRE

Q: What does proactive SRE mean to you?

A: Proactive SRE means finding reliability risks before they become customer-impacting incidents. I use trends in SLIs, resource saturation, error-budget consumption, capacity, recurring alerts and incident patterns to identify problems early. Then I turn those signals into engineering work such as capacity changes, automation, better alerting or resilience testing.

Q: Reactive vs proactive SRE?

A: Reactive SRE starts when an incident has already happened and focuses on mitigation and recovery. Proactive SRE looks for leading indicators before the incident, such as rising latency, increasing memory usage, growing queue depth or repeated pod restarts. I want the incident data to feed proactive work so the same class of failure becomes less likely.

Q: How would you identify an impending reliability problem?

A: I would look for trends rather than only threshold breaches. Examples are steadily increasing latency, error-budget burn, database connection growth, disk growth, memory pressure, pod restarts or increasing deployment failure rate. I would correlate those signals with traffic growth and recent changes, then forecast capacity or investigate the underlying bottleneck.

Q: How do SLOs help proactive SRE?

A: SLOs turn reliability into measurable targets. I can track error-budget burn and identify services that are consistently close to violating their objectives. That helps prioritize engineering work based on customer-facing reliability rather than simply fixing whichever alert happens to be loudest.

Q: What is error-budget burn?

A: Error-budget burn describes how quickly a service is consuming its allowed unreliability. A short, severe incident can burn the budget very quickly, while a slow degradation can consume it over time. I would use burn-rate signals to trigger earlier investigation or deployment controls before the service reaches an SLO breach.

Q: How would you use Prometheus for proactive SRE?

A: I'd collect service, Kubernetes and infrastructure metrics and build recording rules for important SLIs. Then I'd use PromQL for error rate, latency, saturation, capacity and burn-rate calculations. Grafana dashboards would show trends and operational context, while Alertmanager would route actionable alerts. The goal is early detection with low alert noise.

3. Prometheus

Q: Explain Prometheus from an SRE perspective.

A: Prometheus is a time-series monitoring system that collects metrics, commonly by scraping HTTP metrics endpoints. I use it to measure service health, infrastructure behavior and SRE indicators. PromQL lets me calculate rates, percentiles and aggregations, while recording rules can precompute expensive or frequently used queries.

Q: What is a Prometheus exporter?

A: An exporter exposes metrics in a Prometheus-readable format for systems that do not natively expose Prometheus metrics. Examples include node-level metrics or database metrics. Prometheus scrapes the exporter endpoint and stores the resulting time series.

Q: What is PromQL?

A: PromQL is Prometheus's query language. I use it to filter and aggregate time-series data and calculate operational signals such as request rate, error rate and resource utilization. For example, I can use rate over a counter to calculate requests per second and aggregate by service or status code.

Q: How do you calculate API error rate in Prometheus?

A: I'd typically use the rate of failed requests divided by the rate of total requests over a suitable time window. The exact metric names depend on the application instrumentation. I'd then visualize the result in Grafana and alert only when the error rate represents meaningful service degradation.

Q: How do you monitor Kubernetes with Prometheus?

A: I'd collect cluster and workload metrics covering nodes, pods, containers, kube-state information and application metrics. I would monitor CPU, memory, restarts, scheduling failures, request latency and error rates. I'd combine infrastructure signals with application SLIs so an engineer can move from symptom to root cause quickly.

Q: What makes a Prometheus alert good?

A: It should be actionable, have an appropriate severity and ideally represent customer impact or meaningful risk. I avoid paging on every transient metric fluctuation. I use sustained conditions, SLO-based alerts and clear labels or annotations so the responder understands what is happening and where to look.

4. Grafana

Q: How do you design a Grafana SRE dashboard?

A: I start with customer-facing SLIs: availability, latency, traffic and errors. Then I add saturation and infrastructure context such as CPU, memory, disk, Kubernetes health and dependency metrics. I include deployment markers and useful links to logs and traces. The dashboard should support an incident investigation rather than just display a large collection of graphs.

Q: What would you put on an executive reliability dashboard?

A: I'd keep it focused on availability, SLO compliance, error-budget burn, incident trends, latency and major capacity risks. I would avoid low-level infrastructure details unless they directly explain reliability. The dashboard should make it easy to see whether reliability is improving or deteriorating over time.

Q: How do Grafana and Prometheus work together?

A: Prometheus stores and queries time-series metrics, while Grafana provides visualization and dashboards. Grafana sends PromQL queries to Prometheus and renders the results as graphs, tables or other panels. I use this combination to create operational views around SLIs, SLOs, capacity and incidents.

5. OpenTelemetry

Q: What is OpenTelemetry?

A: OpenTelemetry is an open-source observability framework for generating, collecting and exporting telemetry such as traces, metrics and logs. I see it as an instrumentation and telemetry pipeline rather than a visualization platform. It helps standardize how applications emit observability data across services and environments.

Q: Why would you use OpenTelemetry in a microservices platform?

A: In a distributed system, a request can cross many services and dependencies, so individual logs are often insufficient. OpenTelemetry gives us distributed traces with context propagation so we can follow a request across those services. That makes it much easier to identify where latency or errors are introduced.

Q: Explain an OpenTelemetry trace.

A: A trace represents the journey of one request through a distributed system. It contains spans, where each span represents a unit of work such as an API handler, database query or downstream HTTP call. Trace context is propagated between services so the spans can be connected into one request path.

Q: What is a span?

A: A span represents one operation within a trace and typically contains timing, attributes, status and relationships to other spans. For example, an HTTP request span could contain the route and status, while a child span could represent a PostgreSQL query. This lets me identify where time is being spent.

Q: How does OpenTelemetry help incident troubleshooting?

A: I can correlate a customer request with application spans and dependency calls instead of looking at isolated logs. If latency increases, traces can show whether the time is spent in the application, network or database. I can then combine that evidence with Prometheus metrics and logs to narrow the problem quickly.

Q: OpenTelemetry vs Prometheus?

A: Prometheus is primarily a metrics collection and time-series monitoring system. OpenTelemetry provides standardized instrumentation and collection for traces, metrics and logs and can export telemetry to different backends. In a modern SRE stack, they can complement each other: Prometheus for metrics and OpenTelemetry for broader telemetry and distributed tracing.

Q: What is the OpenTelemetry Collector?

A: The Collector is a vendor-neutral component that receives telemetry, processes it and exports it to observability backends. It can perform batching, filtering, enrichment and routing. This separates application instrumentation from the specific backend and gives the platform more control over the telemetry pipeline.

6. Correlation: Prometheus + Grafana + OpenTelemetry

Q: How would you troubleshoot high latency using all three?

A: I'd start in Grafana with the latency SLI and determine when the degradation began. Prometheus metrics would tell me whether the issue is correlated with traffic, CPU, memory, saturation or error rate. Then I'd use OpenTelemetry traces to identify which service or dependency is contributing most of the latency. Finally I'd inspect correlated logs and recent changes before mitigating.

Q: How do metrics, logs and traces complement each other?

A: Metrics tell me that a problem exists and quantify its scale. Logs provide detailed event and error context. Traces show the path of an individual request through a distributed system. I use all three together because no single signal usually explains a complex production incident completely.

Q: How would you reduce observability noise?

A: I'd remove redundant metrics and non-actionable alerts, standardize labels and avoid high-cardinality dimensions unless they provide real value. I'd use recording rules for common queries and sample traces where appropriate. Most importantly, I would make alerts SLO- or customer-impact oriented so the on-call engineer gets actionable signals rather than noise.

7. Chaos + Observability Scenario Questions

Q: The application has 99.99% availability but chaos testing exposes a dependency failure. What do you do?

A: I would treat the chaos result as a resilience gap even if historical availability looks excellent. I'd document the hypothesis and failure mode, quantify the potential customer impact and determine whether existing timeouts, retries, fallback or redundancy are sufficient. Then I'd implement the mitigation and rerun the experiment to prove recovery behavior.

Q: During chaos testing, error rate rises but users are not impacted. Is that a failure?

A: Not necessarily. I'd first determine whether the errors are internal and whether they affect an SLO or customer-facing path. If the system recovers within the defined resilience boundary, the experiment may have validated expected behavior. If the errors reveal unnecessary retries, resource pressure or poor alerting, I would still capture those as improvement opportunities.

Q: A pod is killed and Kubernetes recreates it, but latency spikes for 30 seconds. What would you investigate?

A: I'd check pod readiness behavior, startup time, load-balancer endpoint propagation and whether traffic was sent to the pod before it was actually ready. I'd correlate Prometheus latency and pod metrics with OpenTelemetry traces. I'd also check whether the application needs connection warm-up or cache initialization. The goal is to remove the recovery-induced latency rather than simply proving the pod restarted.

8. Practical Prometheus / Linux / Kubernetes Commands

Q: Prometheus: check a simple request rate

A: rate(http_requests_total[5m])

Q: Prometheus: approximate error rate

A: sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

Q: Kubernetes: inspect events during an incident

A: kubectl get events --sort-by=.lastTimestamp
kubectl get pods -A
kubectl describe pod
kubectl logs --previous

Q: Kubernetes: check resource pressure

A: kubectl top nodes
kubectl top pods -A
kubectl describe node

Q: Linux: check CPU, memory and disk quickly

A: uptime
free -h
df -h
top

9. Client-Level Follow-up Questions

Q: Why should we run chaos experiments if Kubernetes already self-heals?

A: Self-healing mechanisms are assumptions until they are validated under realistic failure conditions. Kubernetes may recreate a pod, but the complete customer path could still experience latency, connection exhaustion or dependency failures. Chaos testing validates the whole system's behavior, not just whether a container restarts.

Q: What is the difference between monitoring and observability?

A: Monitoring is primarily about predefined signals and known failure conditions. Observability is the ability to understand internal system behavior from emitted telemetry, especially for unexpected failures. I use monitoring for reliable detection and observability for deeper diagnosis.

Q: What would you alert on: CPU or SLO?

A: I would generally prioritize customer-impacting SLO signals for paging. CPU can be an important diagnostic or capacity signal, but high CPU does not always mean customers are affected. I may alert on resource saturation when it is a reliable leading indicator of imminent impact.

Q: How would you prove that a reliability improvement worked?

A: I'd define the expected improvement before making the change. Then I'd compare the relevant SLI, incident rate, error-budget burn, recovery time or failure behavior before and after the change. For resilience changes, I prefer rerunning the same chaos experiment and verifying that the system now meets the expected recovery objective.

Q: What would you do if Prometheus itself becomes unavailable?

A: I'd treat observability as a production dependency and design it for resilience. Depending on the architecture, I'd use redundant Prometheus instances, durable or remote storage and independent alerting paths. I would also make sure critical application health isn't dependent on a single monitoring component. The observability stack needs its own availability and recovery strategy.

10. Rapid-Fire Cheat Sheet

Chaos: Hypothesis → steady state → controlled failure → observe → abort/rollback → analyze → remediate → rerun.

Proactive SRE: SLOs → error-budget burn → trends → capacity → recurring incidents → reliability backlog.

Prometheus: scrape → labels → PromQL → recording rules → Alertmanager → actionable alerts.

Grafana: SLI/SLO dashboard → infrastructure → dependencies → deployment markers → logs/traces links.

OpenTelemetry: instrumentation → context propagation → spans/traces → Collector → backend.

Incident: Impact → Triage → Metrics/Logs/Traces → Isolate → Mitigate → Recover → RCA → Prevent.

Kubernetes: Pods → Events → Describe → Logs → Nodes/Resources → Service/Ingress/Network → Config/Secrets → Recent Changes.

AWS network: DNS → VPC/Subnet/CIDR → Routes → SG/NACL → VPN/TGW → ALB/NLB → EC2/EKS → Container → App → PostgreSQL.

AI-first: Specification → structured prompt → generate → test → security review → validate → human ownership.

Senior-level rule: Don't stop at “the pod restarted” or “the alert fired.” Explain whether the customer SLO was protected, why recovery worked or failed, and what you changed to prevent recurrence.

11. Final Interview Positioning

If asked about a chaos or observability product you have not used directly, do not invent experience. Use: **“I haven't worked extensively with that specific tool, but my approach would be…”** and then explain the experiment, telemetry, safety controls and validation. For Prometheus, Grafana, Kubernetes and production SRE concepts, stay anchored to your actual hands-on experience.